

Chapter 11

Chip Boot Control

11.1 Overview

Chip boot process and some chip functions are determined on power-on or hardware reset using strapping pins and eFuse bits. The following functionality can be determined:

- chip boot mode
- enable or disable of ROM messages printing
- source of JTAG signals

ESP32-P4 has five strapping pins:

- GPIO34
- GPIO35
- GPIO36
- GPIO37
- GPIO38

During Chip Reset (see Chapter [10 Reset and Clock](#)), hardware samples and stores the voltage level of strapping pins as strapping bit of “0” or “1” in latches, and holds these bits until the chip is powered down. Software can read the latch status (strapping value) from [GPIO_STRAPPING](#).

11.2 Functional Description

This section provides description of the chip functions and the patterns of the strapping pins and eFuse values to invoke each function.

Notice:

Only documented patterns should be used. If an undocumented pattern is used, it may trigger unexpected behaviors.

11.2.1 Default Configuration

By default, GPIO35 is connected to the chip’s internal pull-up resistor. If GPIO35 is not connected or is connected to an external high-impedance circuit, the internal weak pull-up determines the default input level of this strapping pin (see Table [11.2-1](#)).

Table 11.2-1. Default Configuration of Strapping Pins

Strapping Pin	Default Configuration
GPIO34	Floating
GPIO35	Pull-up
GPIO36	Floating
GPIO37	Floating
GPIO38	Floating

To change the strapping bit values, users can apply external pull-down/pull-up resistors, or use host MCU GPIOs to control the voltage level of these pins when powering on ESP32-P4. After the reset is released, the strapping pins work as normal-function pins.

11.2.2 Boot Mode Control

The values of GPIO35, GPIO36, GPIO37 and GPIO38 at reset determine the boot mode after the reset is released. Table 11.2-2 shows the strapping pin values of GPIO35, GPIO36, GPIO37 and GPIO38 and the associated boot modes.

Table 11.2-2. Boot Mode Control

Boot Mode	GPIO35	GPIO36	GPIO37	GPIO38
SPI Boot mode (default)	1	x ¹	x	x
Joint Download Boot mode ²	0	1	x	x
SPI Download Boot mode ³	0	0	0	1
Invalid Combination ⁴	0	0	1	x
	0	0	0	0

¹ x: values that have no effect on the result and can therefore be ignored.

² Joint Download Boot mode: Joint Download Boot mode supports the following download methods:

- USB-Serial-JTAG Download Boot
- UART Download Boot
- SPI Slave Download Boot
- USB 2.0 OTG Download Boot

³ SPI Download Boot mode: GPIO37 and GPIO38 need to be reserved only when using SPI Download Boot mode. GPIO37 and GPIO38 are floating by default and are in a high-impedance state at reset.

⁴ Invalid Combination: This combination can trigger unexpected behavior and should be avoided.

In SPI Boot mode, the ROM bootloader loads and executes the program from SPI flash to boot the system. SPI Boot mode can be further classified as follows:

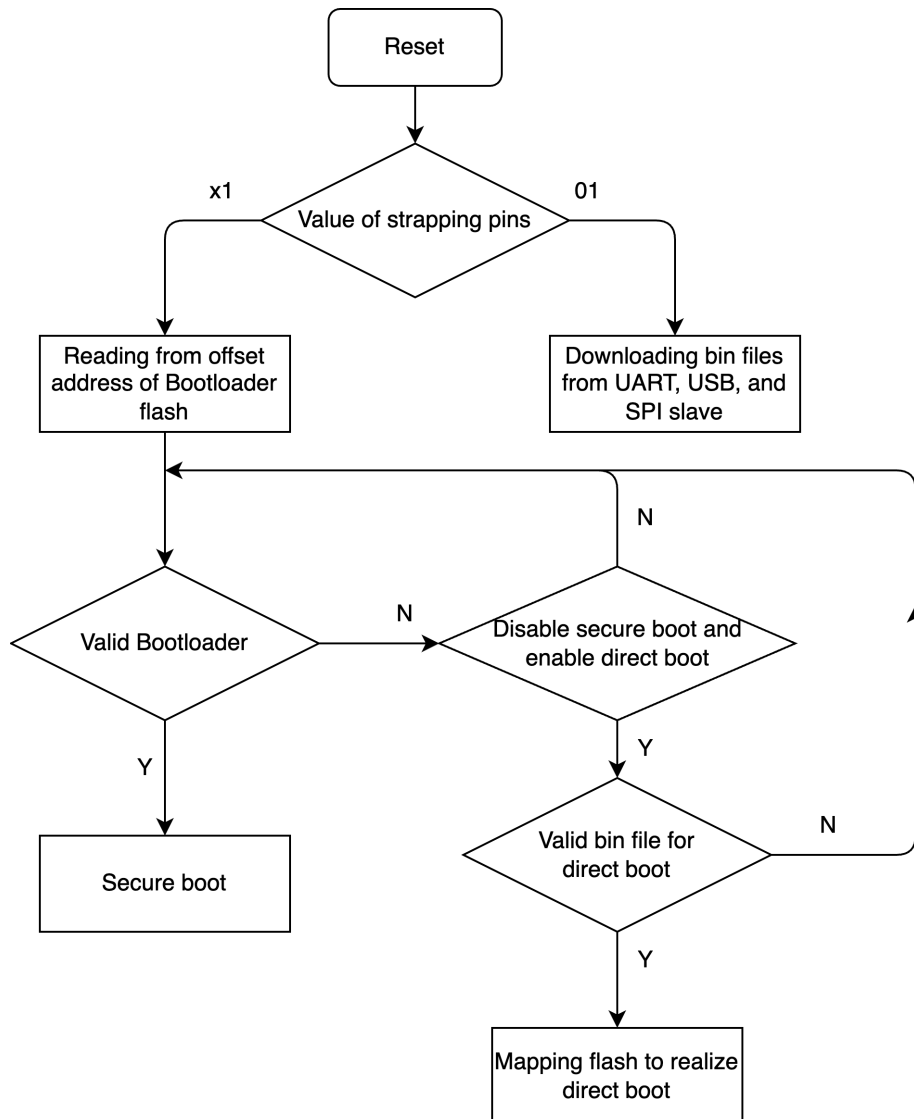
- Normal flash Boot: supports Secure Boot. The ROM bootloader loads the program from flash into L2MEM and executes it. In most practical scenarios, this program is the 2nd stage bootloader, which later boots the target application.

- Direct Boot: does not support Secure Boot and programs run directly from flash. To enable this mode, make sure that the first two words of the bin file downloaded to flash are 0xaedb041d. For more detailed process, see Figure 11.2-1.

In Joint Download Boot mode, users can download binary files into flash using UART0, SPI slave, USB 2.0 OTG or USB Serial/JTAG interface. It is also possible to download binary files into L2MEM and execute it from L2MEM.

In SPI Download Boot mode, users can download binary files into flash using SPI interface. It is also possible to download binary files into L2MEM and execute it from L2MEM.

Figure 11.2-1 shows the detailed boot flow of the chip.



Note: “x1” and “01” are combination values of strapping pins GPIO35 and GPIO36, see Table 11.2-2.

Figure 11.2-1. Chip Boot Flow

The following eFuse bits allows controlling boot mode behaviors:

- [EFUSE_DIS_FORCE_DOWNLOAD](#)

If this eFuse is 0 (default), software can force switch the chip from SPI Boot mode to Joint Download

Boot mode by setting register LPSYSREG_FORCE_DOWNLOAD_BOOT and triggering a CPU reset. If this eFuse is 1, LPSYSREG_FORCE_DOWNLOAD_BOOT is disabled.

- [EFUSE_DIS_DOWNLOAD_MODE](#)

If this eFuse is 1, Joint Download Boot mode is permanently disabled.

- [EFUSE_ENABLE_SECURITY_DOWNLOAD](#)

If this eFuse is 1, Joint Download Boot mode only allows reading, writing, and erasing plaintext flash and does not support any L2MEM or register operations. Ignore this eFuse if Download Boot mode is disabled.

- [EFUSE_DIS_DIRECT_BOOT](#)

If this eFuse is 1, Direct Boot would be disabled in SPI Boot Mode.

USB Serial/JTAG Controller can also force switch the chip to Joint Download Boot mode from SPI Boot mode, and vice versa. For detailed information, please refer to Chapter [51 USB Serial/JTAG Controller \(USB_SERIAL_JTAG\)](#).

You can set eFuse bit EFUSE_DIS_USB_SERIAL_JTAG_DOWNLOAD_MODE to disable USB Serial/JTAG Controller from force switching to Joint Download Boot mode.

11.2.3 ROM Messages Printing Control

During early SPI Boot process, the messages by the ROM code can be printed to:

- (Default) UART0 and USB Serial/JTAG controller
- UART0
- USB Serial/JTAG controller

[EFUSE_UART_PRINT_CONTROL](#) and GPIO36 control ROM messages printing to **UART0** as shown in Table 11.2-3 ROM Message Printing Control.

Table 11.2-3. ROM Message Printing Control

eFuse ¹	GPIO36	ROM Code Printing
0	x ²	Always enabled
1	0	Enabled
	1	Disabled
2	0	Disabled
	1	Enabled
3	x	Always disabled

¹ eFuse: [EFUSE_UART_PRINT_CONTROL](#)

² x: values that have no effect on the result and can therefore be ignored.

[EFUSE_DIS_USB_SERIAL_JTAG_ROM_PRINT](#) controls the printing to **USB Serial/JTAG controller**. When this bit is 1, printing to USB Serial/JTAG controller is disabled. When this bit is 0, and USB Serial/JTAG controller is enabled via EFUSE_DIS_USB_SERIAL_JTAG, ROM messages can be printed to USB Serial/JTAG controller.

Note that if [EFUSE_DIS_USB_SERIAL_JTAG_ROM_PRINT](#) is set to 0 to print to USB, but USB Serial/JTAG Controller has been disabled, then ROM messages will not be printed to USB Serial/JTAG Controller.

11.2.4 JTAG Signal Source Control

GPIO34 controls the source of JTAG signals during the early boot process. This GPIO is used together with [EFUSE_DIS_PAD_JTAG](#), [EFUSE_DIS_USB_JTAG](#), and [EFUSE_JTAG_SEL_ENABLE](#). See Table 11.2-4.

Table 11.2-4. JTAG Signal Source Control

eFuse 1 ^a	eFuse 2 ^b	eFuse 3 ^c	GPIO34	Signal Source
0	0	0	x ^d	JTAG signals come from USB Serial/JTAG Controller.
		1	0	JTAG signals come from corresponding pins MTDI, MTCK, MTMS, and MTDO.
			1	JTAG signals come from USB Serial/JTAG Controller.
0	1	x	x	JTAG signals come from corresponding pins MTDI, MTCK, MTMS, and MTDO.
1	0	x	x	JTAG signals come from USB Serial/JTAG Controller.
1	1	x	x	JTAG is disabled.

^a eFuse 1: [EFUSE_DIS_PAD_JTAG](#)

^b eFuse 2: [EFUSE_DIS_USB_JTAG](#)

^c eFuse 3: [EFUSE_JTAG_SEL_ENABLE](#)

^d x: values that have no effect on the result and can therefore be ignored.